

Constraint Satisfaction Problem

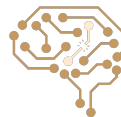
Part One



Arash Maroriyad
6 Ordibehesht 1405

CE 417-Artificial
Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188



Recap

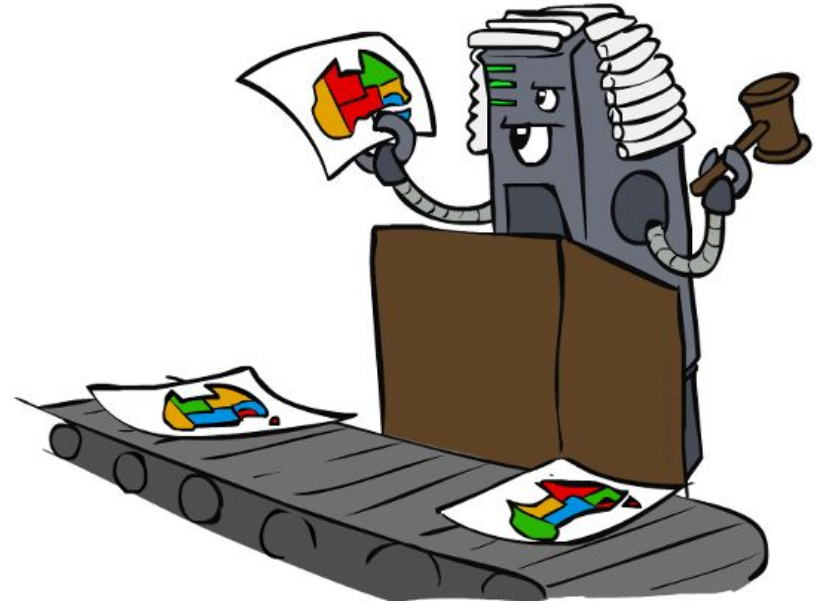
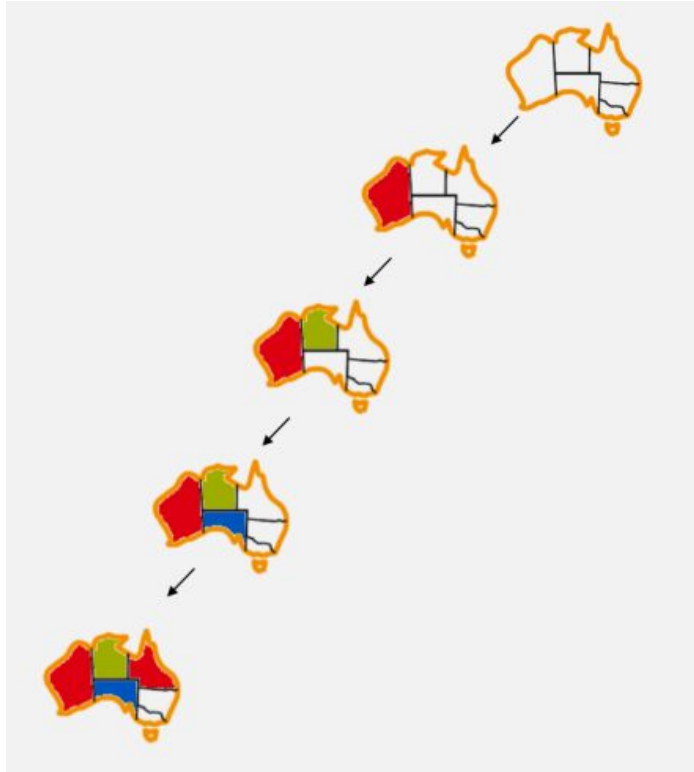
- Uninformed Search
 - BFS, DFS, IDS, and UCS
- Informed Search & Heuristics
 - Greedy and A*
- Local Search
 - Hill Climbing, Simulated Annealing, Beam Search, Genetic Algorithm



Partial State

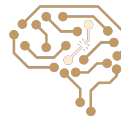
VS

Complete State

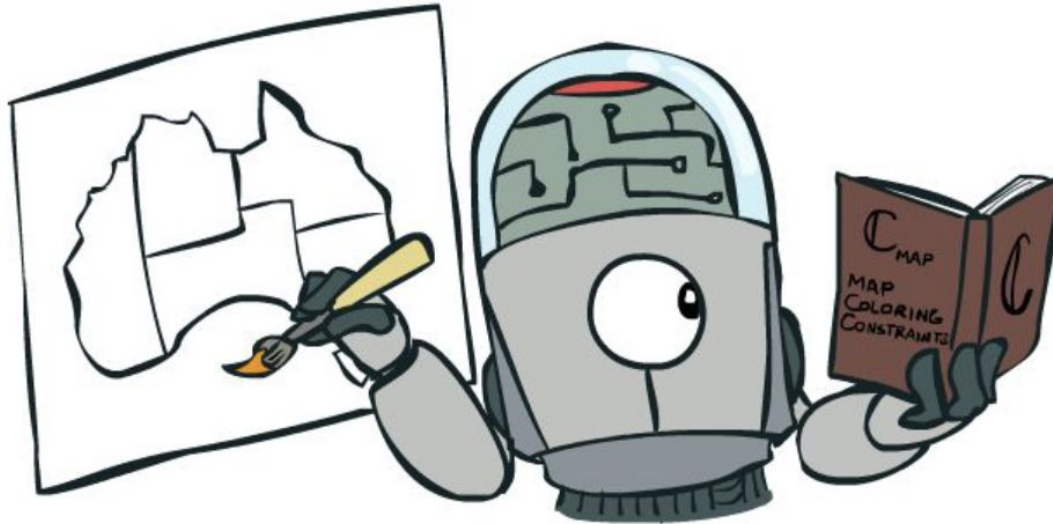


Search Problem: Planning v.s. Identification

- **Planning:** sequences of actions
 - The path to the goal is the important thing
 - Paths have various costs, depths
 - Heuristics give problem-specific guidance
- **Identification:** assignments to variables
 - The goal itself (assignment) is important, not the path
 - All paths at the same depth (for some formulations)
 - CSPs are a specialized class of identification problems



This Session: Constraint Satisfaction Problem (CSP)

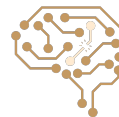
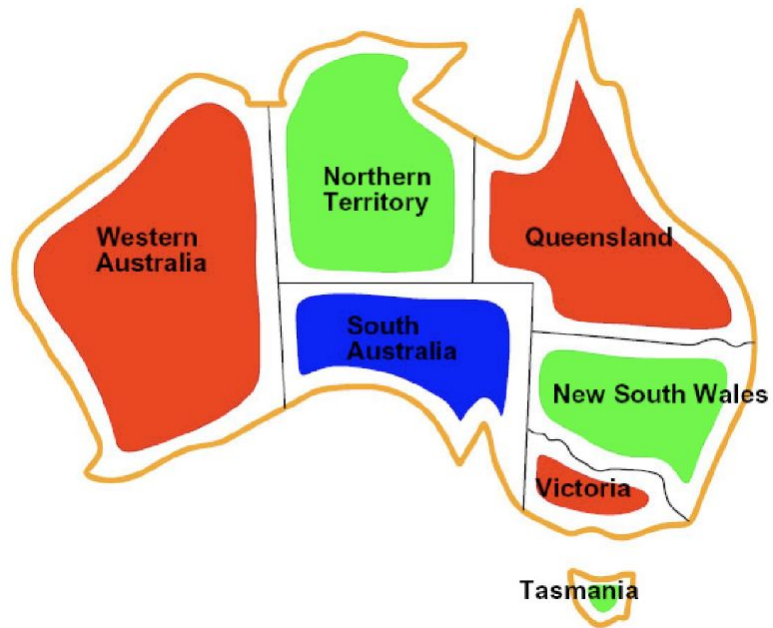


Constraint Satisfaction Problem (CSP)

- A special subset of search problems
- State is defined by variables X_i with values from a domain D
(sometimes D depends on i)
- Goal test is a set of constraints specifying allowable combinations of values for subsets of variables

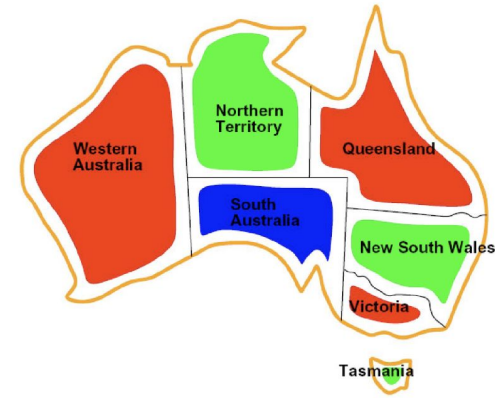


Map Coloring Example

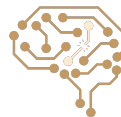
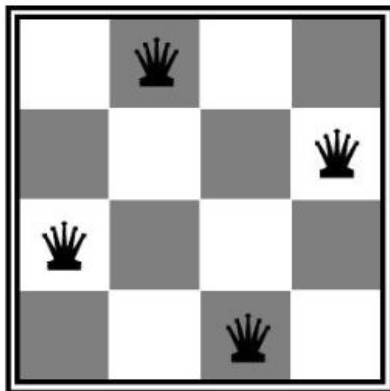


Map Coloring Example (cont.)

- **Variables:** WA, NT, Q, NSW, V, SA, T
- **Domain:** {Red, Green, Blue}
- **Constraint:** Adjacent regions must have different colors
 - **Implicit:** $WA \neq NT$
 - **Explicit:** $(WA, NT) \in \{(Red, Blue), (Red, Green), \dots\}$
- **Solutions** are assignments satisfying all constraints

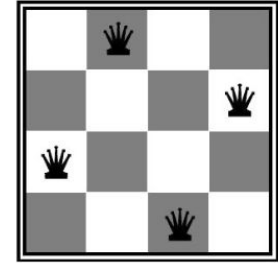


N-Queen Example



N-Queen Example: Formulation I

- Variables: X_{ij}
- Domains: $\{0, 1\}$
- Constraints: None of the queens attack each other!



$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

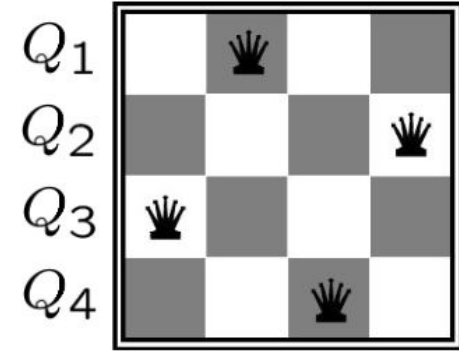
$$\forall i, j, k \quad (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\sum_{i,j} X_{ij} = N$$



N-Queen Example: Formulation II

- Variables: Q_k
- Domains: $\{1, 2, 3, \dots, N\}$
- Constraints: None of the queens attack each other!

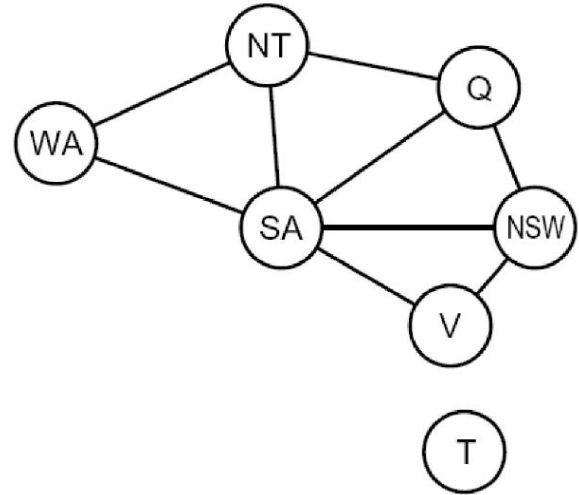
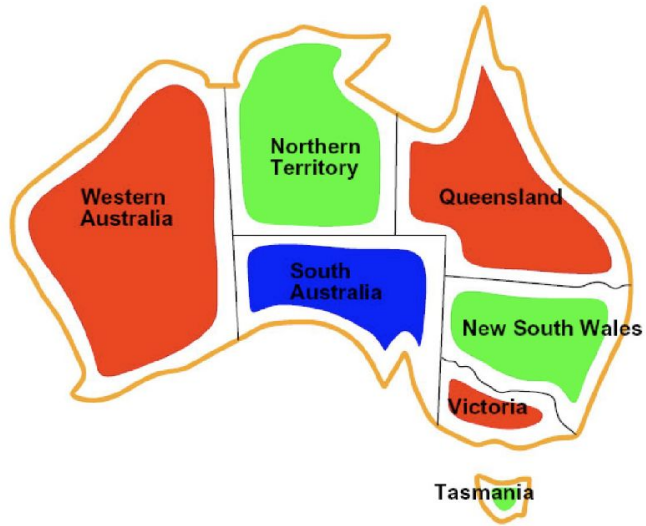


Implicit: $\forall i, j \text{ non-threatening}(Q_i, Q_j)$

Explicit: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$



Constraint Graph



Constraint Graph (cont.)

- **Binary CSP:** each constraint relates (at most) two variables
- **Binary constraint graph:** nodes are variables, arcs show constraints
- CSP algorithms use the graph structure to speed up search.
 - E.g., Tasmania is an independent subproblem



Cryptarithmic Puzzle Example

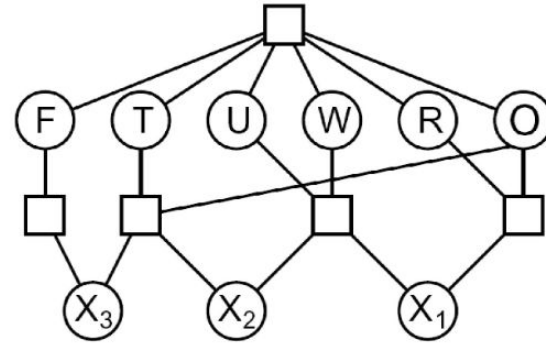
- Variables: $F T U W R O X_1 X_2 X_3$
- Domain: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Constraints:

$\text{alldiff}(F, T, U, W, R, O)$

$$O + O = R + 10 \cdot X_1$$

...

$$\begin{array}{r} T W O \\ + T W O \\ \hline F O U R \end{array}$$



Sudoku Example

- Variables: X_{ij}
- Domain: $\{1, 2, 3, \dots, 9\}$
- Constraints:
 - 9-way alldiff for each row, column, and region
 - Some squares already filled in (unary constraints)

					8			4
	8	4		1	6			
			5			1		
1		3	8			9		
6		8				4		3
		2			9	5		1
		7			2			
			7	8		2	6	
2			3					



Characteristics of CSP: Discrete Variables

- Finite domains:
 - Size d means $O(d^n)$ complete assignments
 - E.g., Boolean CSPs, including Boolean satisfiability (NP-complete)
- Infinite domains (integers, strings, etc.)
 - E.g., job scheduling, variables are start/end times for each job
 - Linear constraints solvable, nonlinear undecidable



Characteristics of CSP: Constraints

- **Unary** constraints involve a single variable, e.g.: SA != Green
- **Binary** constraints involve pairs of variables, e.g.: SA != WA
- **Higher-order** constraints involve 3 or more variables
- Preferences (**soft** constraints):
 - E.g., red is better than green
 - Often representable by a cost for each variable assignment
 - Gives constrained optimization problems



Real World CSPs

- Scheduling problems: e.g., when can we all meet?
- Timetabling problems: e.g., which class is offered when and where?
- Assignment problems: e.g., who teaches what class
- Hardware configuration
- Transportation scheduling
- Circuit layout



How Can We Solve CSPs?



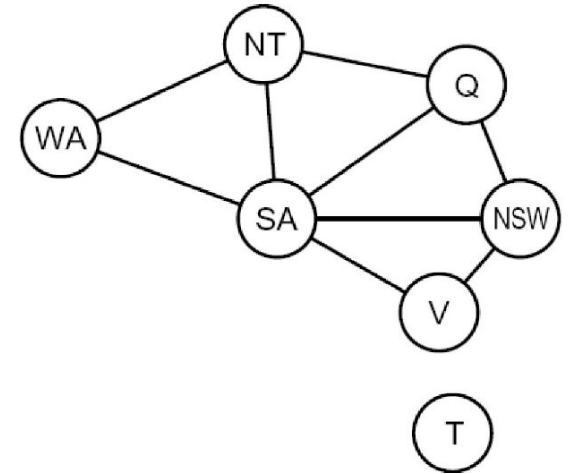
Solving SCP: Tree Search

- **States** defined by the values assigned so far (partial assignments)
- **Initial state:** the empty assignment, $\{\}$
- **Action:** assign a value to an unassigned variable
- **Goal test:** the current assignment is complete and satisfies all constraints



Solving SCP: Tree Search - BFS

- Root: Empty assignment, $\{\}$
- Sweep over partial assignments to 1 variable:
 - $\{WA = R\}, \{WA = B\}, \{WA = G\}, \{NT = R\}, \dots$
- Then partial assignments to 2 variables:
 - $\{WA = R, NT = R\}, \{WA = R, NT = B\}, \dots$
- BFS has to go through whole tree!

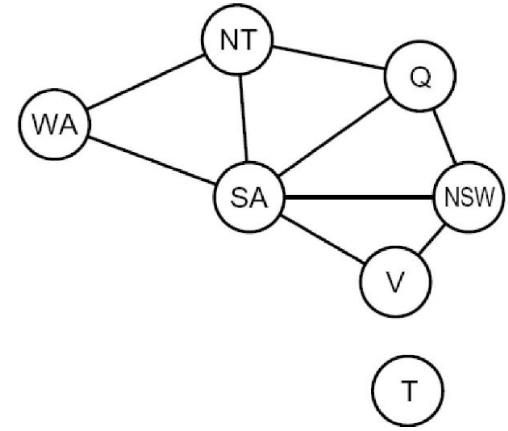


All solutions are at the bottom = bad!



Solving SCP: Tree Search - DFS

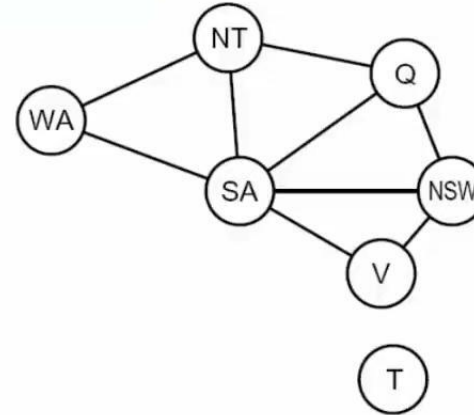
- Perhaps more sensible?
- At least it starts by checking complete solutions
- Let's check the demo!



Solving SCP: Tree Search - DFS Demo

Search Methods

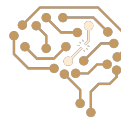
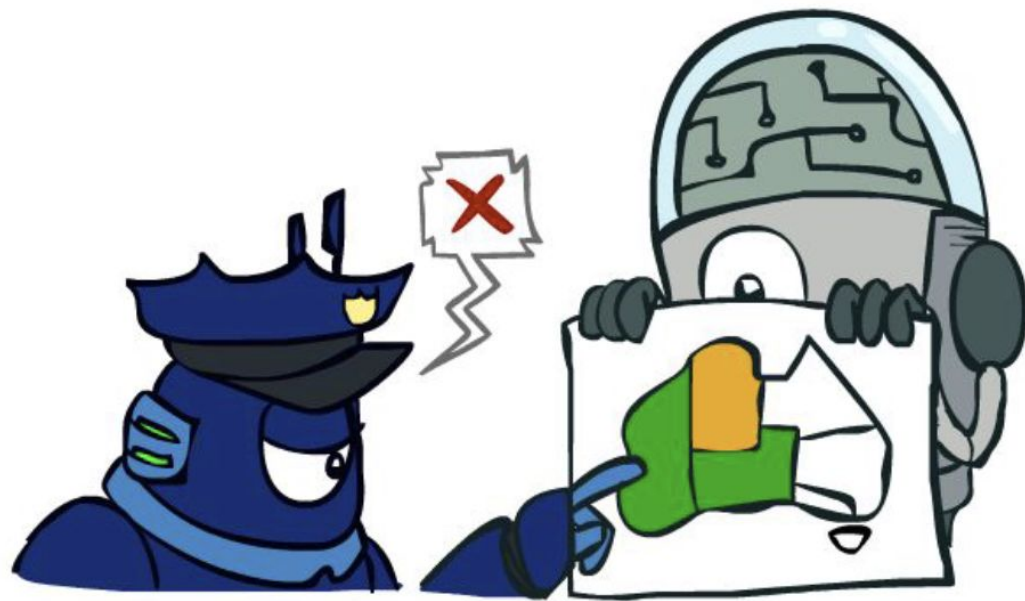
- What would BFS do?
- What would DFS do?



[demo: dfs]



Backtracking Search



Backtracking Search

- A kind of DFS with the two following characteristics.
- One variable at a time, in a fix **ordering**:
 - Variable assignments are commutative, so fix ordering
 - I.e., [WA = red then NT = green] same as [NT = green then WA = red]
 - Only need to consider assignments to a single variable at each step
- **Check constraints** as you go
 - Consider only values which do not conflict with previous assignments
 - Might have to do some computation to check the constraints
 - Incremental goal test



- 



Backtracking Search Algorithm

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```



Backtracking Search Demo

beta.cs188.org/exercises/csps/forward_checking/forward_checking.html

Graph: Simple

Algorithm: Backtracking

Ordering:

- ☒ None
- ☐ MRV
- ☐ MRV with LCV

Filtering:

- ☒ None
- ☐ Forward Checking
- ☐ Arc Consistency

Speed:

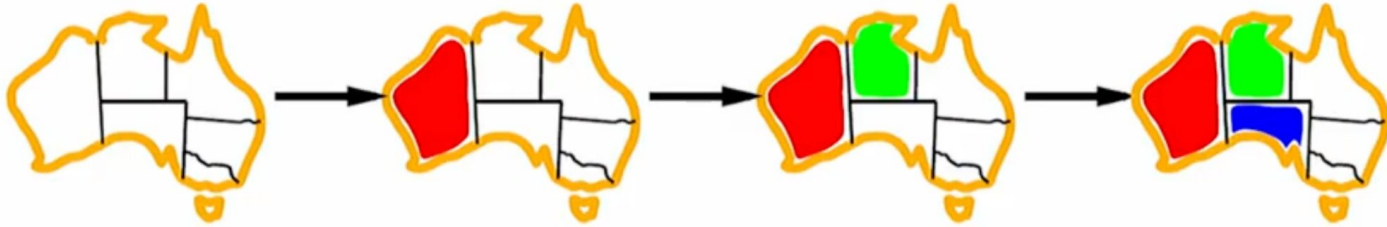
Speedup: 1 x Frame Delay: 700

Reset Prev Pause Next Play Faster



Ordering Heuristics: Which Variable First?

- **Variable Ordering:** Minimum remaining values (MRV)
 - Choose the variable with the **fewest legal left values** in its domain



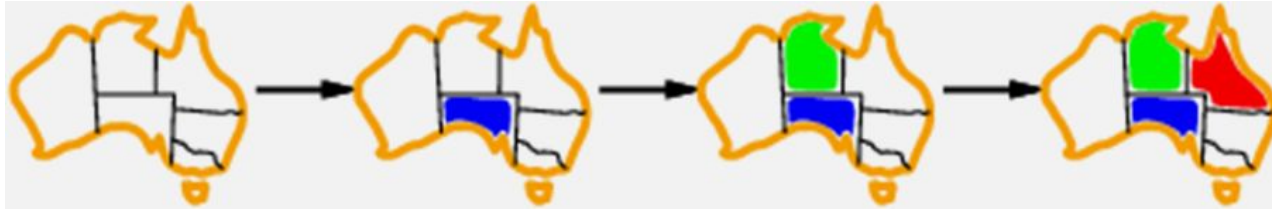
- Why min rather than max?

Also called “most constrained variable” and “Fail-fast” ordering



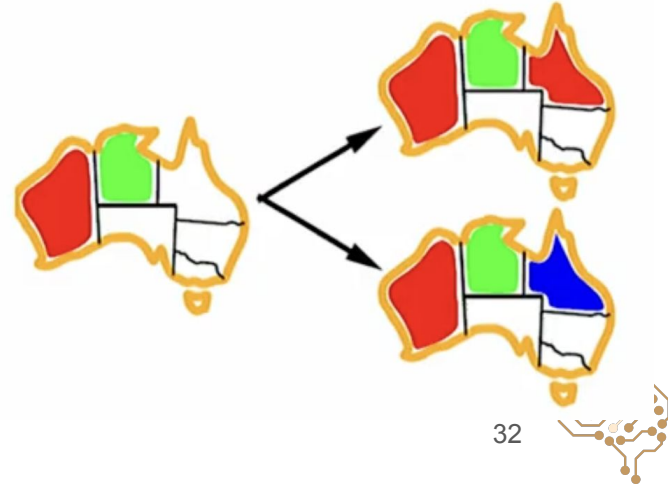
Ordering Heuristics: Which Variable First?

- **Variable Ordering:** Degree heuristic
- Choose the variable with the most constraints on remaining variables
- Tie-breaker among MRV variables



Ordering Heuristics: Which Value First?

- **Value Ordering:** Least Constraining Value
- Given a choice of variable, choose the least constraining value
- I.e., the one that rules out the fewest values in the remaining variables
- It takes extra computation to determine!
- Why the least rather than the most?

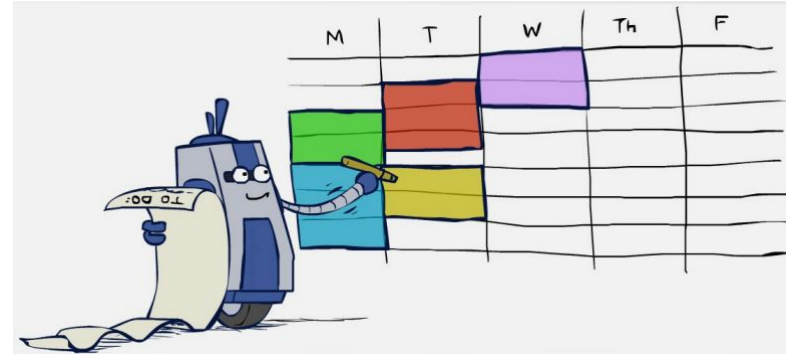


Up To Now ...

- DFS
- Backtracking = DFS + Variable/Value Ordering + Constraining Check
- How to Improve Backtracking?
- Can we detect inevitable failure early?



Next Session: Filtering & Structure



$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$

